

Transition Path Methods for Heterogeneous Agent Models

Rafael Lincoln

March 11, 2026

Abstract

This document provides the mathematical foundations behind the transition path solver implemented in `hetmacro.transition`. We cover two methods for computing perfect foresight transition dynamics after an MIT shock: (i) backward forward shooting with damped iteration, and (ii) root finding via Broyden's method. The exposition follows the style of the numerical appendix in the Macro Bible and is self contained.

Contents

1	Setup: MIT Shocks and Transition Dynamics	2
1.1	The Household Problem Along the Transition	2
1.2	Distribution Dynamics	2
1.3	Market Clearing	3
1.4	Timing Convention	3
2	Method 1: Backward Forward Shooting	3
2.1	The Role of Damping	4
2.2	The EGM Backward Step	5
2.3	The Forward Operator	6
2.4	Resource Constraint Diagnostic	6
3	Method 2: Root Finding via Broyden's Method	6
3.1	Formulation as a Root Finding Problem	6
3.2	Newton's Method and the Jacobian Problem	7
3.3	Broyden's Method: Rank 1 Jacobian Approximation	7
3.4	Comparison of Methods	8
4	Application: TFP Shock in the Aiyagari Model	8
5	Implementation Notes	9

1 Setup: MIT Shocks and Transition Dynamics

Consider a heterogeneous agent economy in steady state at $t = 0$. At $t = 1$, an unexpected (MIT) shock hits the economy. After the shock is realized, agents have perfect foresight over the entire future path of the economy. We seek to compute the transition path from the initial steady state back to the terminal steady state.

1.1 The Household Problem Along the Transition

Let a denote asset holdings and z idiosyncratic productivity. Along the transition, the household problem at date t is:

$$V_t(a, z) = \max_{c, a'} \left\{ u(c) + \beta \sum_{z'} \Pi(z' | z) V_{t+1}(a', z') \right\} \quad (1)$$

subject to the budget constraint:

$$c + a' = (1 + r_t)a + y_t(z), \quad a' \geq \underline{a} \quad (2)$$

where r_t is the interest rate at date t , $y_t(z) = w_t z$ is labor income, and Π is the Markov transition matrix for the idiosyncratic state. The key difference from the stationary problem is that the value function, prices, and policies are all time varying.

Terminal condition. At $t = T$ (the terminal date), we impose that the economy has returned to its terminal steady state:

$$V_T(a, z) = V^{\text{ss}}(a, z) \quad (3)$$

This pins down the backward recursion. The horizon T must be chosen large enough so that this boundary condition is innocuous.

1.2 Distribution Dynamics

Let $D_t(a, z)$ denote the joint distribution of assets and productivity at date t . Given a policy function $a'_t(a, z)$ and the Markov transition matrix Π , the distribution evolves forward as:

$$D_{t+1}(a', z') = \sum_z \Pi(z' | z) \sum_a \mathbf{1}\{a'_t(a, z) = a'\} D_t(a, z) \quad (4)$$

In practice, when policies do not land exactly on grid points, we use lottery weights (linear interpolation on the asset grid) to distribute mass across adjacent grid points.

Initial condition. At $t = 0$, the distribution is the initial steady state distribution:

$$D_0(a, z) = D^{\text{ss}}(a, z) \quad (5)$$

1.3 Market Clearing

In the Aiyagari model, the aggregate capital stock equals mean asset holdings:

$$K_t = \sum_{a,z} a \cdot D_t(a, z) \quad (6)$$

Given aggregate capital K_t and the exogenous shock path $\{Z_t\}_{t=0}^{T-1}$, prices are determined by firm optimality:

$$r_t = Z_t \alpha K_t^{\alpha-1} - \delta \quad (7)$$

$$w_t = Z_t (1 - \alpha) K_t^\alpha \quad (8)$$

The transition path problem is to find a capital path $\{K_t\}_{t=0}^{T-1}$ such that market clearing (6) holds at every date, given that household policies and distributions are consistent with the prices implied by this capital path.

1.4 Timing Convention

The timing follows the standard discrete time convention:

- $t = 0$: shock hits; agents re-optimize immediately.
- $D_0 = D^{\text{ss}}$: the pre-shock steady state distribution (assets are predetermined).
- $K_t = \sum_{a,z} a \cdot D_t(a, z)$: capital at time t equals mean assets that agents *bring into* period t .
- $a'_t(a, z)$: savings chosen at t , which become assets at $t + 1$.
- Backward iteration: $t = T - 1, T - 2, \dots, 0$.
- Forward iteration: $t = 0, 1, \dots, T - 1$.

2 Method 1: Backward Forward Shooting

The shooting method solves the transition path by iterating on the aggregate capital path. The idea is simple: guess a capital path, compute the implied household behavior, check whether the resulting distribution is consistent with the guessed capital, and update.

Algorithm 1 (Backward Forward Shooting with Damped Iteration).

Inputs: Initial steady state $(\bar{K}, \bar{r}, D^{\text{ss}}, V_a^{\text{ss}})$, terminal steady state, shock path $\{Z_t\}_{t=0}^{T-1}$, damping parameter $\lambda \in (0, 1)$, tolerance $\varepsilon > 0$.

Initialize: Set $K_t^{(0)} = \bar{K}$ for all t (or use linear interpolation between initial and terminal steady state capital).

Step 1. Compute prices. For each $t = 0, \dots, T - 1$:

$$r_t^{(n)} = Z_t \alpha \left(K_t^{(n)}\right)^{\alpha-1} - \delta, \quad w_t^{(n)} = Z_t (1 - \alpha) \left(K_t^{(n)}\right)^\alpha$$

Step 2. Backward iteration. Starting from the terminal marginal value V_a^{ss} at $t = T$, iterate backward using the Endogenous Grid Method (EGM):

$$\text{For } t = T - 1, T - 2, \dots, 0 : \quad V_{a,t}, a'_t, c_t = \text{EGM} \left(V_{a,t+1}, \Pi, \mathcal{A}, y_t, r_t, \beta, \sigma^{-1} \right)$$

where EGM denotes a single step of the endogenous grid method, which exploits the Euler equation:

$$u'(c_t(a, z)) = \beta(1 + r_{t+1}) \sum_{z'} \Pi(z' | z) u'(c_{t+1}(a'_t(a, z), z')) \quad (9)$$

This produces a sequence of policy functions $\{a'_t, c_t\}_{t=0}^{T-1}$.

Step 3. Forward iteration. Starting from $D_0 = D^{ss}$, iterate the distribution forward using the time varying policies:

$$\text{For } t = 0, 1, \dots, T - 1 : \quad D_{t+1} = \mathcal{T}(D_t, a'_t, \Pi)$$

where $\mathcal{T}$ denotes the forward operator that distributes mass using lottery weights on the asset grid. At each date, compute the implied capital:

$$\hat{K}_t = \sum_{a,z} a \cdot D_t(a, z) \quad (10)$$

Step 4. Convergence check. Compute the error:

$$e^{(n)} = \max_{0 \leq t \leq T-1} \left| \hat{K}_t - K_t^{(n)} \right|$$

If $e^{(n)} < \varepsilon$, stop and return $\hat{K}$ as the equilibrium path.

Step 5. Damped update. Update the capital path guess with damping:

$$K_t^{(n+1)} = \lambda K_t^{(n)} + (1 - \lambda) \hat{K}_t \quad (11)$$

Return to Step 1.

2.1 The Role of Damping

Without damping ($\lambda = 0$), the algorithm sets the next guess equal to the implied capital path. This often leads to oscillations: if the guess is too high, the implied path overshoots in the other direction, and so forth. Damping stabilizes convergence by taking a convex combination of the old guess and the new implied path.

The parameter λ controls the trade off between stability and speed:

- λ close to 1: very stable but slow convergence (small steps).
- λ close to 0: fast but potentially unstable (large steps).
- A good default is $\lambda = 2/3$, which corresponds to a step size of $1/3$. This is the value used in Midrigan's problem set 5.

Remark 1 (Damping convention). In the implementation, the *damping* parameter equals λ , the weight on the old guess. Some references (including Midrigan's notes) parametrize the update as:

$$K^{(n+1)} = K^{(n)} + s \left(\hat{K} - K^{(n)} \right)$$

where s is the "step size." The relationship is $s = 1 - \lambda$, so Midrigan's $s = 1/3$ corresponds to $\lambda = 2/3$.

2.2 The EGM Backward Step

Each backward step uses the Endogenous Grid Method of Carroll (2006). Given the continuation marginal value $V_{a,t+1}(a', z)$, the EGM step proceeds as follows:

1. Compute the expected marginal value of savings on the *exogenous* grid of end of period assets $a' \in \mathcal{A}$:

$$\mathbb{E}_t [V_{a,t+1}(a', z')] = \sum_{z'} \Pi(z' | z) V_{a,t+1}(a', z') \quad (12)$$

2. Invert the Euler equation to find consumption on this grid:

$$c_t^*(a', z) = (\beta(1 + r_t) \mathbb{E}_t [V_{a,t+1}(a', z')])^{-\sigma} \quad (13)$$

where σ is the coefficient of relative risk aversion (the inverse of the elasticity of intertemporal substitution σ^{-1}).

3. Use the budget constraint to recover the *endogenous* grid of beginning of period assets:

$$a_t^*(a', z) = \frac{c_t^*(a', z) + a' - y_t(z)}{1 + r_t} \quad (14)$$

4. Interpolate: the previous step produced pairs $(a_t^*(a'_k, z), a'_k)$ on an *endogenous* grid of beginning of period assets. To obtain policies on the *exogenous* grid $\mathcal{A} = \{a_1, \dots, a_N\}$ used by the rest of the algorithm, apply linear interpolation. For each $a_i \in \mathcal{A}$, find the bracketing pair $a_j^* \leq a_i < a_{j+1}^*$ and set

$$a'_t(a_i, z) = a'_j + \frac{a_i - a_j^*}{a_{j+1}^* - a_j^*} (a'_{j+1} - a'_j), \quad (15)$$

and analogously for consumption, $c_t(a_i, z) = c_j^* + \frac{a_i - a_j^*}{a_{j+1}^* - a_j^*} (c_{j+1}^* - c_j^*)$.

5. Enforce the borrowing constraint: if $a < a_t^*(\underline{a}, z)$, set $a'_t(a, z) = \underline{a}$ and $c_t(a, z) = (1 + r_t)a + y_t(z) - \underline{a}$.
6. Update the marginal value using the envelope condition:

$$V_{a,t}(a, z) = (1 + r_t) u'(c_t(a, z)) \quad (16)$$

2.3 The Forward Operator

Given a distribution D_t and a policy a'_t , the forward step computes D_{t+1} . Since policies generally do not land on grid points, we use lottery weights. For a given (a, z) with policy $a'_t(a, z)$, find the bracketing grid points $a_i \leq a'_t(a, z) < a_{i+1}$ and compute the weight:

$$\omega = \frac{a_{i+1} - a'_t(a, z)}{a_{i+1} - a_i} \quad (17)$$

Then distribute the mass $D_t(a, z) \cdot \Pi(z' \mid z)$ across (a_i, z') with weight ω and (a_{i+1}, z') with weight $1 - \omega$.

2.4 Resource Constraint Diagnostic

After convergence, we verify the resource constraint at each date:

$$Y_t - C_t - A_t + (1 - \delta)K_t = 0 \quad (18)$$

where $Y_t = Z_t K_t^\alpha$ is output, $C_t = \sum_{a,z} c_t(a, z) D_t(a, z)$ is aggregate consumption, and $A_t = \sum_{a,z} a'_t(a, z) D_t(a, z)$ is aggregate savings (next period's capital). The residual should be near machine precision ($\sim 10^{-10}$).

3 Method 2: Root Finding via Broyden's Method

An alternative to the shooting method is to cast the transition path as a system of non-linear equations and solve it directly using a quasi Newton method. This approach can converge faster (in fewer iterations), though each iteration is more expensive.

3.1 Formulation as a Root Finding Problem

Define the residual function $F : \mathbb{R}^T \rightarrow \mathbb{R}^T$ as follows. Given a candidate capital path $\mathbf{K} = (K_0, K_1, \dots, K_{T-1})$:

1. Compute prices $\{r_t, w_t\}$ from $\mathbf{K}$ and $\{Z_t\}$.
2. Run the backward iteration to obtain policies $\{a'_t, c_t\}$.
3. Run the forward iteration to obtain implied capital $\{\hat{K}_t\}$.
4. Return the residual:

$$F(\mathbf{K}) = \hat{\mathbf{K}} - \mathbf{K} \quad (19)$$

The equilibrium capital path satisfies $F(\mathbf{K}^*) = \mathbf{0}$. Steps 1 through 3 are identical to a single iteration of the shooting method.

3.2 Newton's Method and the Jacobian Problem

Newton's method would update the guess as:

$$\mathbf{K}^{(n+1)} = \mathbf{K}^{(n)} - \left[J_F(\mathbf{K}^{(n)}) \right]^{-1} F(\mathbf{K}^{(n)}) \quad (20)$$

where $J_F \in \mathbb{R}^{T \times T}$ is the Jacobian of F . Computing and inverting this Jacobian is prohibitively expensive for large T : each column requires a full backward forward pass (to compute one finite difference), so the cost is $O(T)$ evaluations of F , and inversion costs $O(T^3)$.

3.3 Broyden's Method: Rank 1 Jacobian Approximation

Broyden's method avoids computing the full Jacobian by maintaining a rank 1 approximation $B^{(n)} \approx J_F(\mathbf{K}^{(n)})$ that is updated at each step.

Algorithm 2 (Broyden's Method for Transition Paths).

Inputs: Initial guess $\mathbf{K}^{(0)}$, tolerance $\varepsilon > 0$.

Initialize: Set $B^{(0)} = I$ (identity matrix) or a finite difference approximation.

Step 1. Evaluate residual. Compute $F(\mathbf{K}^{(n)})$ via one backward forward pass.

Step 2. Convergence check. If $\|F(\mathbf{K}^{(n)})\|_\infty < \varepsilon$, stop.

Step 3. Compute search direction. Solve for the Newton step:

$$\Delta \mathbf{K}^{(n)} = - \left[B^{(n)} \right]^{-1} F(\mathbf{K}^{(n)}) \quad (21)$$

Step 4. Line search. Optionally perform a line search (Armijo condition) to find a step size α_n such that:

$$\|F(\mathbf{K}^{(n)} + \alpha_n \Delta \mathbf{K}^{(n)})\| < \|F(\mathbf{K}^{(n)})\|$$

Step 5. Update guess.

$$\mathbf{K}^{(n+1)} = \mathbf{K}^{(n)} + \alpha_n \Delta \mathbf{K}^{(n)}$$

Step 6. Update Jacobian approximation. Define:

$$\mathbf{s}^{(n)} = \mathbf{K}^{(n+1)} - \mathbf{K}^{(n)}$$

$$\mathbf{y}^{(n)} = F(\mathbf{K}^{(n+1)}) - F(\mathbf{K}^{(n)})$$

Apply the Broyden rank 1 update:

$$B^{(n+1)} = B^{(n)} + \frac{(\mathbf{y}^{(n)} - B^{(n)} \mathbf{s}^{(n)})(\mathbf{s}^{(n)})^\top}{\|\mathbf{s}^{(n)}\|^2} \quad (22)$$

Return to Step 1.

Why Broyden works well here. The key insight is that the Jacobian of the transition path residual F has a special structure. A change in K_t affects prices at date t , which propagates through the backward forward pass in a smooth, monotone fashion. The rank 1 updates capture this structure well after a few iterations, leading to superlinear convergence in practice.

3.4 Comparison of Methods

	Shooting	Root finding (Broyden)
Cost per iteration	1 backward forward pass	1 backward forward pass
Convergence rate	Linear	Superlinear
Typical iterations	25–40	10–20
Robustness	Very robust with damping	Sensitive to initial guess
Tuning parameter	Damping λ	Line search
Memory	$O(T)$	$O(T^2)$ (Jacobian approx.)

In practice, shooting is the default choice due to its robustness. Root finding is useful when speed matters (shorter T , good initial guess) or when damping is difficult to tune.

4 Application: TFP Shock in the Aiyagari Model

Consider the standard Aiyagari (1994) economy with a Cobb Douglas production function $Y_t = Z_t K_t^\alpha$. At $t = 1$, an unexpected TFP shock hits:

$$\begin{aligned} \log Z_0 &= 0, & \log Z_1 &= 0.10 \\ \log Z_t &= 0.95 \log Z_{t-1}, & t &\geq 2 \end{aligned} \tag{23}$$

The price function that maps aggregate capital and time to household inputs is:

$$\begin{aligned} r_t &= Z_t \alpha K_t^{\alpha-1} - \delta \\ w_t &= Z_t (1 - \alpha) K_t^\alpha \\ y_t(z) &= w_t \cdot z \end{aligned} \tag{24}$$

Calibration. The baseline parameters are: $\beta = 0.99$, $\gamma = 1.5$ (CRRA), $\rho_z = 0.99$, $\sigma_\varepsilon = 0.10$, $\alpha = 0.35$, $\delta = 0.02$, with 11 Rouwenhorst income states and 501 asset grid points up to $\bar{a} = 2500$.

Results. The shooting method with $\lambda = 2/3$ converges in approximately 34 iterations. Capital rises above the steady state (agents save more in response to higher TFP and wages), then decays back as the shock fades. The resource constraint residual is near machine precision ($\sim 10^{-10}$). The root finding method (Broyden) matches the shooting solution to within $\sim 10^{-8}$ in capital.

5 Implementation Notes

The solver in `hetmacro.transition` is model agnostic. The user provides:

1. Steady state dictionaries (from `household_steady_state`) containing the distribution D , marginal value V_a , policies, and aggregates.
2. A price function `price_fn(K, t) -> dict` that maps aggregate capital and time to household inputs $\{r, y, w\}$.

Key design choices.

- The backward step defaults to EGM (`backward_egm`), but a custom `backward_fn` can be provided to use any solver (VFI, Howard improvement, Euler iteration) via the `make_backward_fn` wrapper.
- Capital at time t is computed as $K_t = \sum_{a,z} a \cdot D_t(a, z)$ (mean assets in the distribution), not as aggregate savings from policies. This avoids a timing mismatch between the guess and the implied path.
- The income argument to the steady state solver must be $y = w \cdot z$ (income levels), not raw productivity z . This ensures consistency between the steady state and the transition price function.